

# DSAA 4040 Course Project Proposal

## Design and Deployment of a Cloud-Native Online Bookstore on Kubernetes

Zhiyuan Wang

Yukun Li

Spring 2026

### 1. Project Overview

This project aims to design and implement a cloud-native online bookstore system on Kubernetes. The system will consist of three main components: a frontend, a backend API, and a database. It will support the full workflow from application development and containerization to Kubernetes deployment, service exposure, basic monitoring, and small-scale performance testing.

The project is positioned as an engineering-oriented system project. In addition to building a functional bookstore application, we aim to demonstrate practical cloud-native capabilities such as containerized deployment, configuration management, autoscaling, health checking, and basic observability.

### 2. Functional Design

#### 2.1. Core Business Functions

The system will first provide a complete and runnable bookstore application with the following core functions:

- **Book Browsing and Search.** Users can view the list of books and search by title, keyword, or category. Each book entry will display basic information such as author, price, and stock.
- **Shopping Cart Management.** Users can add books to the cart, update item quantities, remove items, and view the current cart content. Since cart-related operations are frequent interactive actions, this part of the system will prioritize fast response and smooth user interaction.
- **Order Placement and Persistence.** Users can place orders based on the shopping cart, and the system will write order data into the database. Since order placement is a critical write path, this part of the system will place more emphasis on correctness and consistency, especially for operations such as order creation and stock update.

#### 2.2. Cloud-Native Functions

Based on the business functions above, the project will also include the following cloud-native features:

- **Containerized Deployment.** The frontend, backend, and database will be packaged as separate containers to keep the system modular and easier to deploy.
- **Kubernetes-Based Service Deployment.** The system will be deployed on Kubernetes, where different components run as separate services and communicate through internal service discovery.
- **Configuration Management and Service Exposure.** The project will use ConfigMap and Secret to manage regular configuration and sensitive information, and use Ingress to expose the application externally.
- **Auto-Scaling.** The backend service will support automatic scaling under workload changes, so that the system can better handle increased request pressure.
- **Health Checks.** The system will include basic health checking mechanisms to determine whether a service is alive and ready to receive traffic.
- **Basic Monitoring.** The project will observe basic runtime metrics such as resource usage, Pod status, and replica changes, which will help with debugging and system evaluation.
- **Small-Scale Performance Testing.** The system will be tested under lightweight request pressure to observe response behavior and runtime performance under different load levels.

### 3. Technical Architecture

The system will follow a three-tier architecture:

- **Frontend:** responsible for page rendering and user interaction;
- **Backend API:** responsible for business logic and interface handling;
- **Database:** responsible for persistent storage of books, shopping cart data, and orders.

The overall request path is:

**Browser → Frontend → Backend API → Database**

This architecture keeps the system modular and makes it easier to connect deployment, scaling, health checking, and monitoring mechanisms in later stages.

### 4. Tools and Environment

This project will be implemented in a lightweight environment recommended by the course. The expected tool choices are as follows:

- **Application Development:** common frontend technologies and a lightweight backend web framework;
- **Data Storage:** a relational database;
- **Containerization:** Docker;
- **Cluster Environment:** one of Minikube, Docker Desktop Kubernetes, or K3s;
- **Kubernetes Features:** Deployment, Service, ConfigMap, Secret, Ingress, autoscaling, and health-check-related mechanisms;

- **Monitoring and Testing:** basic resource monitoring tools and lightweight load testing tools.

## 5. Implementation Plan

The project will be completed in the following stages:

1. **Requirement and Structure Design** Define the business workflow, system modules, data model, and API structure.
2. **Core Function Implementation** Implement book browsing and search, shopping cart management, and order placement with database persistence.
3. **Containerization and Deployment** Package the frontend, backend, and database as containers, and complete the initial Kubernetes deployment and service configuration.
4. **Cloud-Native Feature Integration** Add configuration management, Ingress, autoscaling, health checks, and basic monitoring.
5. **Testing and Demonstration** Conduct functional testing and small-scale performance testing, and prepare deployment documentation and presentation materials.

## 6. Expected Deliverables

The expected deliverables of this project include:

- source code and configuration files of the bookstore system;
- containerization-related files and Kubernetes deployment manifests;
- a README with setup and deployment instructions;
- the final project report;
- proposal, presentation slides, and demo materials.

## 7. Conclusion

This project aims to build a functional online bookstore while demonstrating a practical cloud-native deployment workflow on Kubernetes. By combining business functionality with autoscaling, health checking, monitoring, and performance testing, the project is expected to provide a complete engineering-oriented implementation aligned with the course objectives.